Pricing Baku Homes From Scratch

A NumPy decision tree and Pegasos SVM on bina.az listings

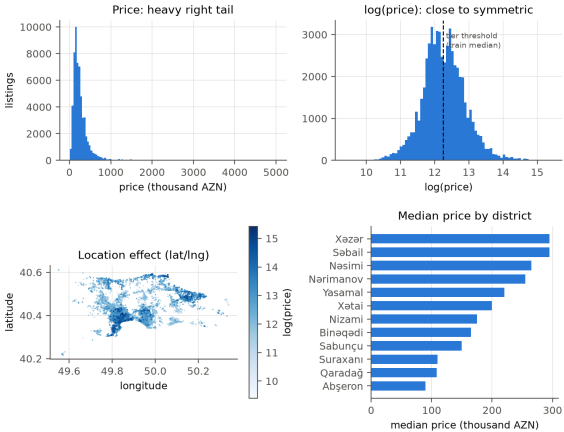
TODO: team member names

MLE-AI-201 Machine Learning — AI Academy

Final project demo

Problem and data

- bina.az Kaggle dump: 100,775 scraped records, 51 columns in Azerbaijani
- Cleaned: **57,422** unique Baku residential listings
- **Task A:** regress log(price)
- **Task B:** premium vs standard, split at the *train* median (209,000 AZN)
- Split 70/15/15, stratified, seed 42: 40,196 / 8,613 / 8,613



Cleaning, features and leakage

Cleaning (every step logged)

- 33k re-scrapes of the same URL removed (newest kept), plus 709 re-posts
- Strings → numbers: “145 m²”, “7 / 9”, “1.3 sot”
- Fixed domain outlier rules (no quantiles ⇒ nothing learned from test)
- Imputation medians and one-hot vocabularies from *train only*

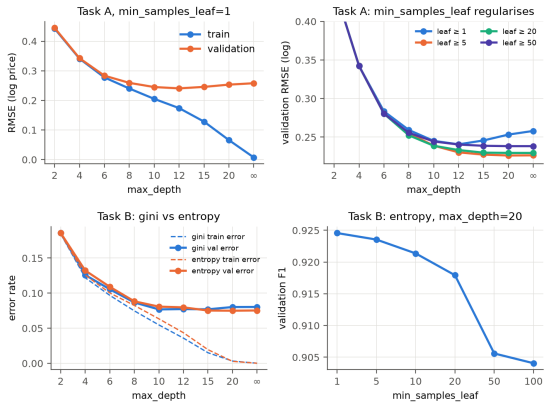
Removed: leakage and identifiers

- `unit_price` = price / area
- `total_price` = price
- owner/shop names, address, ids, URLs
- description (restates the price), `views` (arrives after listing)

99 features: area, rooms, floors, land, lat/lng, distance to centre, amenities, one-hot category / district / neighbourhood

Our decision tree

- Gini, entropy (classification); MSE (regression)
- $\mathrm{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$
- Per feature: sort once, prefix sums ⇒ score *all* thresholds in one vectorised pass
- Stopping: `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease`
- Tests: **identical splits to sklearn** on a toy example; every hyperparameter changes the tree
- Bug found on real data: the midpoint of adjacent floats rounds up ⇒ an empty child

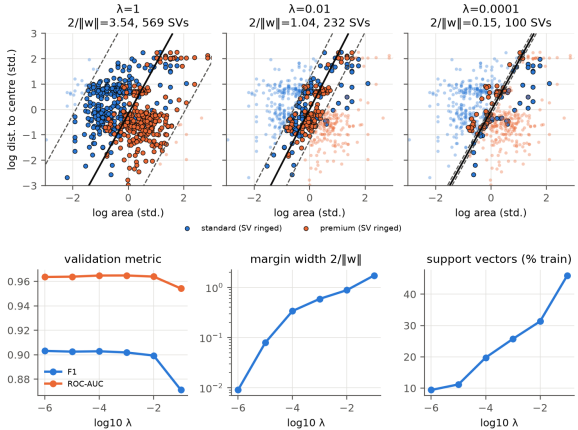


Our Pegasos SVM

$J = \frac{\lambda}{2}\|\mathrm{w}\|^2 + \frac{1}{n}\sum_i \max(0, 1 - y_i \mathrm{w}^\top \mathrm{x}_i)$

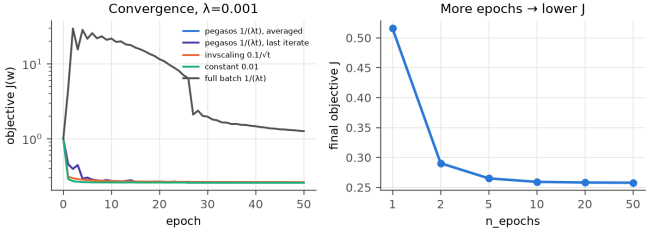
$\mathrm{w} \leftarrow (1 - \eta_t\lambda)\mathrm{w} + \frac{\eta_t}{|B|}\sum_{i\in A} y_i \mathrm{x}_i,\ \eta_t = \frac{1}{\lambda t}$

- Mini-batch, projection, iterate averaging
- Margin $2/\|\mathrm{w}\|$, support vectors $y_i f(\mathrm{x}_i) \le 1$
- Non-linear: **random Fourier features** ($O(nD)$ vs $O(n^2)$ for kernel Pegasos)
- Chosen $\lambda = 0.001$: $\|\mathrm{w}\| = 3.386$, margin 0.591, 25.7% SVs



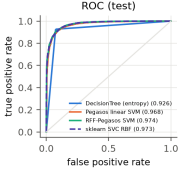
Experimental setup

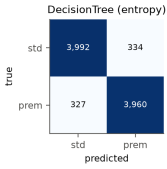
- Tuning on **validation only**; the test set is used once
- Tree A: depth 20, leaf ≥ 5 Tree B: entropy, depth 20, leaf ≥ 1
- SVM: $\lambda = 0.001$ RFF: $\gamma = 0.003$, $\lambda = 1e-05$, $D = 1000$ (most regularised within 0.002 F1 of the best)
- Baselines with **matched** settings: sklearn trees; `Pipeline(StandardScaler, SGDClassifier(hinge, alpha=`λ`))`; SVC linear and RBF with $C = 1/(\lambda n)$ (10k subsample)
- Uncertainty: bootstrap std on test, plus 5-fold CV mean $\pm$ std on train+val

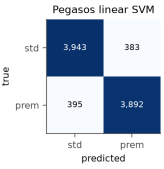


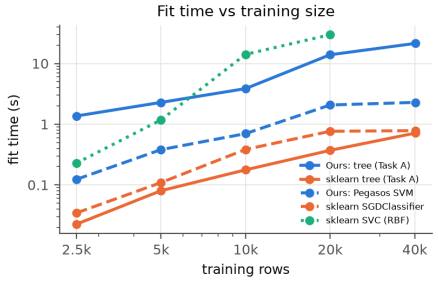
Results

Model	A: RMSE log	A: R^2 log	A: CV RMSE	B: F1	B: AUC	B: CV F1
Baseline: predict train mean	0.636	-0.000	n/a	—	—	—
Ours: DecisionTree (MSE)	0.231	0.869	0.234 ± 0.003	—	—	—
sklearn DecisionTreeRegressor	0.232	0.867	0.233 ± 0.002	—	—	—
Ours: DecisionTree (entropy)	—	—	—	0.923	0.926	0.923 ± 0.003
sklearn DecisionTreeClassifier	—	—	—	0.927	0.930	0.923 ± 0.003
Ours: Pegasos linear SVM	—	—	—	0.909	0.968	0.906 ± 0.003
Ours: RFF-Pegasos SVM	—	—	—	0.913	0.974	0.909 ± 0.002
sklearn SGDClassifier (hinge)	—	—	—	0.907	0.967	0.905 ± 0.003
sklearn SVC linear (10k subsample)	—	—	—	0.904	0.966	0.905 ± 0.003
sklearn SVC RBF (10k subsample)	—	—	—	0.910	0.973	0.908 ± 0.003





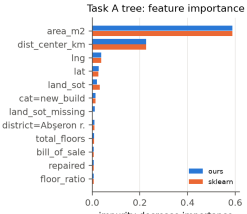


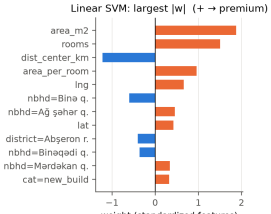


Ours vs scikit-learn: why the small gaps?

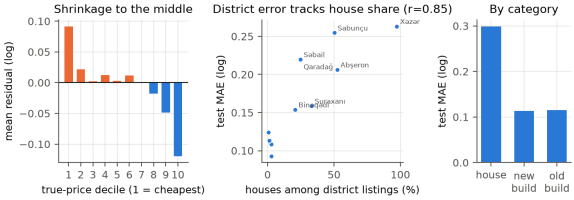
	depth (ours/sk)	leaves (ours/sk)	first 7 splits equal	same test pred.
Task A	20 / 20	5324 / 5337	6/7	93.6%
Task B	20 / 20	2231 / 2239	7/7	98.2%

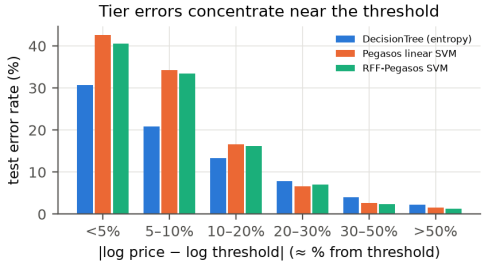
- Same root split, 6–7 of the first 7 splits equal; 93.6% / 98.2% identical test predictions
- Gaps: sklearn uses float32 thresholds; random feature order on ties; divergence after a deep disagreement
- Speed: our tree 17.3 s vs 0.70 s (Cython); our Pegasos 5.0 s





Where the models fail





- **Tails**: leaf means shrink to the middle, so luxury is under-predicted and bargains over-predicted
- **Houses** are much harder than flats; district error tracks house share ($r = 0.85$) more than district size ($r = -0.55$)
- **Tier**: 77% of SVM errors are within ∼20% of the threshold, and many are irreducible
- Limits: asking prices only; one 5-week snapshot; random (not grouped) split; text unused

Who did what and next steps

- TODO: one line per member (see `contribution_report.md`)

Next

- Grouped / temporal split to measure generalisation to new buildings
- Text features from descriptions (with price mentions masked)
- Random forest / boosting from our own trees

Reproduce: `pip install -r requirements.txt && python -m src.get_data && python -m src.run_all`